Administration des bases de données Oracle

Résumé de cours — Chapitres 1 à 5

Table des matières

1 Architecture du serveur Oracle 2
1.1 Serveur, instance et base de données . . . 2
1.2 Instance Oracle : mémoire SGA . . . 2
1.3 Processus d'arrière-plan et PGA . . . 3
1.4 Base de données : le dictionnaire de données . . . 3
1.5 Fichiers de paramètres d'initialisation . . . 4
1.6 Démarrage de la base . . . 5
1.7 Arrêt de la base . . . 5

2 Structures de stockage Oracle 6
2.1 Structures logiques et physiques . . . 6
2.2 Tablespaces . . . 6
2.3 Extents et segments . . . 6
2.4 Gestion de l'espace dans un tablespace . . . 6
2.5 Créer un tablespace . . . 6
2.6 Modifier un tablespace . . . 7
2.7 Supprimer un tablespace . . . 7
2.8 Consulter les informations . . . 7

3 Sécurité utilisateur 7
3.1 Compte utilisateur . . . 7
3.2 Authentification . . . 8
3.3 Créer un utilisateur . . . 8
3.4 Privilèges . . . 8
3.5 Options d'administration . . . 9
3.6 Rôles . . . 9
3.7 Profils . . . 10

4 Sécurité et audit 10
4.1 Sécurité de la base : principe du moindre privilège . . . 10
4.2 Les trois types d'audit . . . 11
4.3 Audit standard . . . 11
4.4 Audit basé sur les données (triggers) . . . 12
4.5 Audit détaillé (FGA) . . . 12

5 Déplacement des données 13
5.1 Oracle Data Pump . . . 13

5.2 SQL*Loader 15

6 Aide-mémoire 17

1 Architecture du serveur Oracle

1.1 Serveur, instance et base de données

Définition (Serveur Oracle)

Un *serveur Oracle* est un SGBD qui fournit une approche intégrée, complète et ouverte de la gestion des informations. Il se compose d'une **instance Oracle** (mémoire + processus) et d'une **base de données Oracle** (fichiers).

Scénario de connexion.

1. Le client contacte le *listener* Oracle en indiquant l'instance voulue (nom de service).
2. Le listener démarre un processus dédié, le *processus serveur*.
3. Le listener renvoie au client un accusé de réception avec l'adresse du processus serveur.
4. Le client établit la connexion avec ce processus serveur dédié.
5. Le processus serveur se connecte à l'instance pour le compte du processus utilisateur (création de la *session*).

Le processus utilisateur n'interagit jamais directement avec le serveur Oracle : c'est le processus serveur qui exécute les demandes et renvoie les résultats.

1.2 Instance Oracle : mémoire SGA

La **SGA** (System Global Area) est une mémoire partagée par tous les processus serveurs et les processus d'arrière-plan.

Zone SGA	**Rôle**
Zones obligatoires	
Shared Pool	Cache du dictionnaire de données (*row cache*) : informations du dictionnaire (comptes, fichiers, segments, privilèges…) réutilisées lors de l'analyse des requêtes. Cache *library* : instructions SQL et code PL/SQL récents, chargés une seule fois pour plusieurs exécutions.
Database buffer cache	Copies des blocs de données lus dans les fichiers de données ; géré par un algorithme **LRU** ; taille du bloc principal : `DB_BLOCK_SIZE`.
Redo log buffer	Enregistre toutes les modifications faites aux blocs de données (entrées de journalisation), pour la **récupération** ; taille : `LOG_BUFFER`.
Zones facultatives	
Large pool, Java pool, Streams pool	Zones mémoire complémentaires.

1.3 Processus d'arrière-plan et PGA

Processus	Nom	Rôle / déclenchement
DBWn	Database Writer	Écrit les blocs modifiés (*dirty*) du cache vers les fichiers de données : point de reprise, seuil de tampons dirty atteint, plus de tampon libre, délai dépassé, tablespace hors ligne ou en lecture seule, DROP/TRUNCATE d'une table, BEGIN BACKUP.
LGWR	Log Writer	Écrit les entrées de journalisation sur disque : à chaque **COMMIT**, quand un tiers du tampon est plein, à 1 Mo de journalisation, toutes les 3 secondes, avant une écriture de DBWn.
SMON	System Monitor	Récupération de l'instance après panne (rejoue les redo logs, ouvre la base, annule les transactions non validées), fusion de l'espace libre, libération des segments temporaires.
PMON	Process Monitor	Nettoie après l'échec d'un processus utilisateur : annule la transaction, libère verrous et ressources, redémarre les répartiteurs interrompus.
CKPT	Checkpoint	Signale les points de reprise à DBWn et met à jour les en-têtes des fichiers de données et les fichiers de contrôle.

Définition (PGA)

La **PGA** (Program Global Area) est une mémoire *privée* créée pour chaque processus serveur et chaque processus d'arrière-plan : zones privées de curseurs, variables attachées (*bind*), informations de session, zone de tri. Elle est libérée à la fin de la session.

1.4 Base de données : le dictionnaire de données

Le dictionnaire de données décrit la base : noms et attributs de tous les objets. Il est créé avec la base, mis à jour à chaque création ou modification d'objet, appartient à l'utilisateur **SYS**, et est stocké dans des tables de base auxquelles on accède *uniquement* par des vues prédéfinies.

Préfixe	Usage
`USER_`	Ce que contient votre schéma (objets dont vous êtes propriétaire)
`ALL_`	Vue étendue : ce à quoi vous pouvez accéder
`DBA_`	Vue de l'administrateur : ce que contient le schéma de chaque utilisateur
`V\$`	Données relatives aux performances (vues dynamiques)

Vue	Contenu
DICTIONARY	Nom et description des tables et vues du dictionnaire
USER_OBJECTS	Tous vos objets : nom, type, date de création, dernière modification, statut (valide / non valide) ; ALL_OBJECTS pour ceux auxquels vous avez accès
USER_TABLES	Informations sur vos tables (TABLE_NAME, TABLESPACE_NAME, ...)
USER_TAB_COLUMNS	Colonnes de vos tables (type, longueur, précision, échelle, NULLABLE, ...)
USER_CONSTRAINTS	Contraintes sur vos tables (CONSTRAINT_TYPE : C, U, P, R ; SEARCH_CONDITION, DELETE_RULE, STATUS)
USER_CONS_COLUMNS	Colonnes impliquées dans des contraintes
USER_VIEWS	Vos vues (VIEW_NAME, TEXT : requête de définition)
USER_SEQUENCES	Vos séquences ; LAST_NUMBER = prochain numéro disponible
USER_SYNONYMS	Vos synonymes

```
SELECT object_name, object_type, created, status FROM user_objects ORDER BY object_type;
SELECT column_name, data_type, data_length, data_precision, data_scale, nullable
FROM user_tab_columns WHERE table_name = 'EMPLOYEES';
SELECT constraint_name, constraint_type, search_condition, r_constraint_name, delete_rule, status
FROM user_constraints WHERE table_name = 'EMPLOYEES';
```

Définition (Vues dynamiques V$)

Elles enregistrent l'activité en cours, sont constamment mises à jour, lues depuis la mémoire et le fichier de contrôle, commencent par V\$ et sont accessibles par un DBA. Exemples : V\$SESSION (sessions en cours), V\$LOGFILE (fichiers journaux), V\$LOG (statut des groupes de journaux).

1.5 Fichiers de paramètres d'initialisation

Une base n'est disponible que lorsque le DBA a démarré l'instance et ouvert la base. Pour démarrer, Oracle lit un fichier de paramètres afin d'allouer la SGA et lancer les processus.

	PFILE (initSID.ora)	**SPFILE** (spfileSID.ora)
Format	Fichier texte	Fichier binaire
Modification	Manuelle (éditeur), effective au démarrage suivant	Ne pas éditer à la main ; géré par le serveur (dynamique)
Particularité	Indispensable sur les postes clients	Pas besoin d'en copier sur les clients distants

```
CREATE PFILE [= 'nom_pfile'] FROM SPFILE [= 'nom_spfile'];
CREATE SPFILE [= 'nom_spfile'] FROM PFILE [= 'nom_pfile'];
```

Paramètres usuels : CONTROL_FILES (fichiers de contrôle), DB_FILES (nombre max de fichiers), PROCESSES (nombre max de processus OS), DB_BLOCK_SIZE, DB_CACHE_SIZE, SGA_TARGET, MEMORY_TARGET, PGA_AGGREGATE_TARGET, SHARED_POOL_SIZE, UNDO_MANAGEMENT.

Paramètres statiques : modifiables seulement dans le fichier de paramètres (ALTER SYSTEM ... SCOPE='SPFILE') ; redémarrage nécessaire. **Paramètres dynamiques** : modifiables base en ligne, au niveau *session* (ALTER SESSION) ou *système* (ALTER SYSTEM).

```
ALTER SYSTEM SET parametre = valeur COMMENT = '...' [SCOPE = MEMORY | SPFILE | BOTH];
ALTER SESSION SET parametre = valeur;
SHOW PARAMETER nom_parametre;
```

À retenir : SCOPE

`MEMORY` : changement immédiat, perdu à l'arrêt. `SPFILE` : pris en compte au prochain démarrage (paramètres statiques). `BOTH` : immédiat *et* persistant ; c'est la valeur par défaut si la base a démarré avec un SPFILE. Avec un PFILE, seule la portée `MEMORY` existe.

La vue `V\$PARAMETER` donne les valeurs courantes (`NAME`, `TYPE`, `VALUE`, `ISDEFAULT`). Sa colonne `ISSYS_MODIFIABLE` indique si le paramètre est statique (`FALSE`), modifiable immédiatement (`IMMEDIATE`) ou en différé (`DEFERRED`) ; `ISSES_MODIFIABLE` concerne la session.

1.6 Démarrage de la base

```
STARTUP [OPEN [READ {ONLY | WRITE}] | MOUNT | NOMOUNT | FORCE | RESTRICT]
```

Option	Effet
`NOMOUNT`	Crée la SGA et démarre les processus d'arrière-plan ; pas d'accès à la base (création de base ou recréation des fichiers de contrôle)
`MOUNT`	Associe la base à l'instance démarrée ; pas d'accès utilisateur
`OPEN`	Ouvre fichiers de données et journaux en ligne ; accès utilisateurs (`READ ONLY` ou `READ WRITE`)
`FORCE`	Effectue un `SHUTDOWN ABORT` puis un redémarrage (si le démarrage normal échoue)
`RESTRICT`	Base ouverte, mais accessible seulement aux utilisateurs ayant `RESTRICTED SESSION` (tuning, import/export) ; levée par `ALTER SYSTEM DISABLE RESTRICTED SESSION`

Passage d'un état à l'autre : `ALTER DATABASE MOUNT`, `ALTER DATABASE OPEN [READ WRITE | READ ONLY]`. Ordre normal : **NOMOUNT → MOUNT → OPEN**.

1.7 Arrêt de la base

À la fermeture, Oracle écrit les changements du buffer cache et du redo log buffer dans les fichiers de données et de journalisation, ferme ces fichiers (les fichiers de contrôle restent ouverts), démonte la base, ferme les fichiers de contrôle, arrête l'instance (libération de la SGA, arrêt des processus d'arrière-plan, fermeture des fichiers trace et alert).

Mode	Nouvelles connexions	Attente des utilisateurs	Transactions en cours	Récupération au démarrage
`NORMAL` (défaut)	refusées	oui (toutes les déconnexions)	—	non
`TRANSACTIONAL`	refusées	fin de la transaction en cours	terminées, puis déconnexion	non
`IMMEDIATE`	refusées	non	SQL en cours non traités ; rollback	non
`ABORT`	refusées	non	non validées annulées ; fichiers non fermés	**oui**

2 Structures de stockage Oracle

2.1 Structures logiques et physiques

Structure logique	Structure physique
Base de données ⊃ tablespaces ⊃ segments ⊃ extents ⊃ blocs de données Oracle	Fichiers de données ⊃ blocs du système d'exploitation

Logiquement, les données sont stockées dans des **tablespaces** : ils n'appartiennent qu'à une seule base, sont constitués d'un ou plusieurs fichiers de données, et divisés en unités de stockage logiques. Physiquement, elles sont dans des **fichiers de données**, qui n'appartiennent qu'à un seul tablespace et une seule base.

2.2 Tablespaces

— Une base a au moins un tablespace : **SYSTEM**, qui contient le dictionnaire de données (Oracle recommande de n'y stocker rien d'autre).
— Un tablespace peut être **online** (données accessibles) ou **offline** ; SYSTEM ne peut jamais être mis hors ligne.
— Avantages pour le DBA : organiser la base (quotas utilisateurs, disponibilité des données via mise hors service ou lecture seule) et améliorer les performances en répartissant le stockage sur plusieurs disques.

2.3 Extents et segments

Définition (Extent / extension)

Suite *contiguë* de blocs, affectée à un type de données (par ex. les enregistrements d'une table) ; le nombre de blocs est fixé par le DBA.

Définition (Segment)

Ensemble d'extents dédié au stockage d'un type d'objet (table, index, ...). Un extent initial est alloué à la création, puis de nouveaux extents sont alloués dynamiquement au besoin.

2.4 Gestion de l'espace dans un tablespace

— **Géré localement** (recommandé) : les extents libres sont gérés dans le tablespace par un *bitmap* ; chaque bit correspond à un bloc et indique s'il est libre ou utilisé.
— **Géré par le dictionnaire** : les extents libres sont gérés par les tables du dictionnaire, mises à jour à chaque allocation ou libération.

2.5 Créer un tablespace

```
CREATE {BIGFILE|SMALLFILE} TABLESPACE nom_tablespace
  [DATAFILE ['nom_fichier'] [SIZE integer {K|M|G|T}]
     [AUTOEXTEND {OFF | ON [NEXT integer {K|M|G|T}] [MAXSIZE {UNLIMITED | integer {K|M|G|T}}]}]
         [,...]]
  [BLOCKSIZE integer [K]]
  [{LOGGING|NOLOGGING}] [{ONLINE|OFFLINE}]
  [extent_management_clause] [DEFAULT storage_clause];

CREATE TABLESPACE TBL01
  DATAFILE 'C:\ORACLE\PRODUCT\10.2.0\ORADATA\ORCL\FDAUTO1.dbf' SIZE 10M
```

```
  AUTOEXTEND ON NEXT 2M MAXSIZE 20M;
```

`LOGGING|NOLOGGING` : les modifications des objets du tablespace sont journalisées ou non. `AUTOEXTEND` : étend automatiquement le fichier de données en cas de saturation, avec une limite éventuelle.

2.6 Modifier un tablespace

```
ALTER TABLESPACE TBL01 ADD DATAFILE 'C:\ORACLE\ORADATA\BDSCO2\TBL01fd03.dbf'
  SIZE 30M AUTOEXTEND ON NEXT 5M MAXSIZE 100M; -- ajouter un fichier
ALTER DATABASE DATAFILE 'C:\ORACLE\ORADATA\BDSCO2\TBL01fd03.dbf' RESIZE 50M; -- redimensionner
ALTER TABLESPACE TBL01 READ ONLY; -- plus d'ecriture (les transactions en cours se terminent)
ALTER TABLESPACE TBL01 READ WRITE;
ALTER TABLESPACE TBL01 ONLINE; -- disponible
ALTER TABLESPACE TBL01 OFFLINE; -- indisponible
```

2.7 Supprimer un tablespace

Commande	Effet
`DROP TABLESPACE nom`	Suppression logique
`DROP TABLESPACE nom AND DATAFILES`	Suppression logique et physique (fichiers supprimés)
`DROP TABLESPACE nom INCLUDING CONTENTS`	Supprime aussi les objets stockés dans le tablespace
`... INCLUDING CONTENTS CASCADE CONSTRAINTS`	Supprime aussi les contraintes d'intégrité

2.8 Consulter les informations

Information	Vues
Tablespaces	`DBA_TABLESPACES`, `V\$TABLESPACE`
Fichiers de données	`DBA_DATA_FILES`, `V\$DATAFILE`
Fichiers temporaires	`DBA_TEMP_FILES`, `V\$TEMPFILE`
Extents / segments	`DBA_EXTENTS`, `DBA_SEGMENTS`
Espace libre restant	`DBA_FREE_SPACE`
Taille de bloc par défaut	paramètre `DB_BLOCK_SIZE`

3 Sécurité utilisateur

3.1 Compte utilisateur

Chaque compte comporte : un nom *unique*, une méthode d'authentification, un tablespace par défaut, des quotas sur un ou plusieurs tablespaces, un tablespace temporaire, un profil, un statut (verrouillé ou expiré). Seuls le **nom** et l'**authentification** sont obligatoires ; le reste a des valeurs par défaut. Le nom ne peut plus être modifié après création. Les informations se consultent dans `DBA_USERS`.

Comptes prédéfinis. SYS : rôle DBA, tous les privilèges avec `ADMIN OPTION`, requis pour démarrage, arrêt et certaines opérations de maintenance, propriétaire du dictionnaire de données. **SYSTEM** : reçoit le rôle DBA.

3.2 Authentification

Mode	Principe	Syntaxe
Système d'exploitation	Privilège SYSDBA ou SYSOPER	`CONNECT / AS SYSDBA`
Fichier de mots de passe	Privilège SYSDBA ou SYSOPER, mots de passe copiés dans un fichier externe lisible même base fermée	`CONNECT user/pwd AS SYSDBA`
Mot de passe (base)	Mot de passe saisi à chaque connexion ; peut expirer immédiatement (changement à la 1re connexion)	`CREATE USER u IDENTIFIED BY pwd`
Externe	Délégué à un service externe (Kerberos, RADIUS, Windows) ou à l'OS ; préfixe `OS_AUTHENT_PREFIX` (ex. `ops\$`)	`CREATE USER u IDENTIFIED EXTERNALLY`
Globale	Via Oracle Internet Directory	`CREATE USER u IDENTIFIED GLOBALLY`

3.3 Créer un utilisateur

```
CREATE USER nom IDENTIFIED BY mot_de_passe
  [DEFAULT TABLESPACE ts] [TEMPORARY TABLESPACE ts]
  [QUOTA {valeur [K|M] | UNLIMITED} ON ts [,...]]
  [PROFILE profil] [PASSWORD EXPIRE] [ACCOUNT {LOCK | UNLOCK}];

CREATE USER scott IDENTIFIED BY pwdscott
  DEFAULT TABLESPACE users QUOTA UNLIMITED ON users PASSWORD EXPIRE;
```

3.4 Privilèges

Définition (Privilège)

Droit d'exécuter un type d'instruction SQL ou d'accéder à un objet d'un autre utilisateur.

— **Privilèges système** : opérations de base de données (créer des tablespaces, des sessions...). Accordés par l'administrateur ou par quelqu'un ayant reçu le droit de les administrer.

— **Privilèges objet** : action particulière sur un objet précis (table, vue, séquence, procédure, fonction, package). Sans privilège, un utilisateur n'accède qu'à ses propres objets. Accordés par le propriétaire, l'administrateur, ou un utilisateur autorisé explicitement.

```
GRANT CREATE SESSION TO HR;
GRANT SELECT, UPDATE(SALARY) ON EMPLOYEES TO SCOTT;
REVOKE CREATE SESSION FROM HR;
REVOKE SELECT, UPDATE(SALARY) ON EMPLOYEES FROM SCOTT;
```

Exemples de privilèges système (le suffixe `ANY` concerne n'importe quel schéma) :

Catégorie	Privilèges
Session	CREATE SESSION, ALTER SESSION, RESTRICTED SESSION
Table	CREATE TABLE, CREATE ANY TABLE, DROP ANY TABLE, SELECT ANY TABLE, INSERT / UPDATE / DELETE ANY TABLE, BACKUP ANY TABLE
Index, cluster	CREATE ANY INDEX, ALTER / DROP ANY INDEX, CREATE CLUSTER
Procédure, trigger	CREATE PROCEDURE, EXECUTE ANY PROCEDURE, CREATE TRIGGER, ALTER ANY TRIGGER
Utilisateur, rôle	CREATE USER, ALTER USER, DROP USER, BECOME USER, CREATE ROLE, GRANT ANY ROLE
Divers	ALTER DATABASE, GRANT ANY PRIVILEGE, CREATE SYNONYM, FORCE TRANSACTION

Privilèges objet : ALTER (table, séquence), DELETE, EXECUTE (procédure, fonction), INDEX, INSERT, REFERENCES (clé étrangère), SELECT, UPDATE.

3.5 Options d'administration

À retenir : déléguer un privilège

WITH ADMIN OPTION (privilège *système*) et WITH GRANT OPTION (privilège *objet*) permettent au bénéficiaire de transmettre le privilège.

Système : si HR reçoit CREATE SESSION WITH ADMIN OPTION et le donne à SCOTT, un REVOKE sur HR **n'est pas transmis** : SCOTT garde le privilège.
Objet : si STOCK reçoit SELECT ON EMPLOYEES WITH GRANT OPTION et le donne à USER1, un REVOKE sur STOCK **est transmis** : USER1 le perd aussi (révocation en cascade).

3.6 Rôles

Définition (Rôle)

Ensemble nommé de privilèges, accordé à des utilisateurs ou à d'autres rôles.

Avantages : gestion simplifiée (on accorde les privilèges au rôle, puis le rôle aux utilisateurs) ; gestion dynamique (modifier un rôle met à jour immédiatement tous ses bénéficiaires) ; disponibilité sélective (un rôle s'active ou se désactive).

Rôle prédéfini	Privilèges
CONNECT	CREATE SESSION, CREATE TABLE, CREATE VIEW, CREATE SYNONYM, CREATE SEQUENCE, CREATE DATABASE LINK, CREATE CLUSTER, ALTER SESSION
RESOURCE	CREATE TABLE, CREATE PROCEDURE, CREATE SEQUENCE, CREATE TRIGGER, CREATE TYPE, CREATE CLUSTER, CREATE INDEXTYPE, CREATE OPERATOR
SCHEDULER_ADMIN	CREATE ANY JOB, CREATE JOB, EXECUTE ANY CLASS, EXECUTE ANY PROGRAM, MANAGE SCHEDULER
DBA	La plupart des privilèges système et plusieurs rôles ; à ne pas accorder à des non-administrateurs
SELECT_CATALOG_ROLE	Aucun privilège système, mais plus de 1600 privilèges objet sur le dictionnaire de données

```
CREATE ROLE "SUPERDBA" IDENTIFIED BY "sup"; -- mot de passe demande a l'activation du role
GRANT ALTER ANY INDEXTYPE TO "SUPERDBA";
GRANT ALTER ANY TABLE TO "SUPERDBA";
```

```
GRANT "DBA" TO "SUPERDBA" WITH ADMIN OPTION;
GRANT "SUPERDBA" TO STOCK WITH ADMIN OPTION;
```

3.7 Profils

Un seul profil est affecté à un utilisateur à un instant donné. Un profil **gère les mots de passe** (toujours actif) et **limite les ressources** (actif seulement si `RESOURCE_LIMIT = TRUE`, valeur par défaut `FALSE`).

Paramètre	Description
Mots de passe	
`FAILED_LOGIN_ATTEMPTS`	Nombre d'échecs de connexion avant verrouillage du compte
`PASSWORD_LOCK_TIME`	Nombre de jours de verrouillage après ces échecs
`PASSWORD_LIFE_TIME`	Durée de vie du mot de passe (jours) avant expiration
`PASSWORD_GRACE_TIME`	Période de grâce (jours) pour changer le mot de passe après expiration
`PASSWORD_REUSE_TIME`	Nombre de jours pendant lesquels un mot de passe ne peut pas être réutilisé
`PASSWORD_REUSE_MAX`	Nombre de changements avant qu'un mot de passe puisse être réutilisé
`PASSWORD_VERIFY_FUNCTION`	Fonction PL/SQL de vérification de complexité ; elle doit appartenir à SYS et renvoyer un booléen
Ressources	
`SESSIONS_PER_USER`	Nombre de sessions concurrentes par compte
`CPU_PER_SESSION`	Temps CPU (centisecondes) maximal par session
`CPU_PER_CALL`	Temps CPU (centisecondes) maximal par requête
`CONNECT_TIME`	Durée de connexion maximale d'une session
`IDLE_TIME`	Durée d'inactivité maximale d'une session

```
CREATE PROFILE developer_profile LIMIT
  SESSIONS_PER_USER 2 CPU_PER_SESSION 10000 CONNECT_TIME 480 IDLE_TIME 60
  FAILED_LOGIN_ATTEMPTS 3 PASSWORD_LIFE_TIME 90 PASSWORD_REUSE_TIME 2
  PASSWORD_VERIFY_FUNCTION my_function;
-- valeurs possibles : integer | UNLIMITED | DEFAULT (fonction | NULL | DEFAULT pour la
    verification)
```

4 Sécurité et audit

4.1 Sécurité de la base : principe du moindre privilège

Un système sécurisé garantit la confidentialité des données : authentifier les utilisateurs, gérer utilisateurs et profils, limiter l'accès aux données et services, surveiller les activités suspectes.

Définition (Principe du moindre privilège)

N'accorder à un utilisateur que les privilèges dont il a réellement besoin pour effectuer sa tâche.

Mesure	Détail
Protéger le dictionnaire	`O7_DICTIONARY_ACCESSIBILITY = FALSE` : empêche les utilisateurs ayant un privilège `ANY TABLE` d'accéder aux tables de base du dictionnaire, et interdit à SYS de se connecter autrement que `SYSDBA`
Révoquer de PUBLIC	Tout utilisateur peut utiliser les privilèges de PUBLIC ; il faut révoquer ceux qui ne sont pas indispensables. Toujours révoquer `EXECUTE` sur `UTL_SMTP` (envoi d'e-mails arbitraires), `UTL_HTTP` (envoi de données vers un site malveillant) et `UTL_FILE` (accès texte aux fichiers de l'OS)
Limiter les administrateurs	Restreindre l'octroi de privilèges système et objet, les connexions `SYSDBA` / `SYSOPER`, et les privilèges de type DBA (`DROP ANY TABLE`)
Authentification à distance	Désactiver : `REMOTE_OS_AUTHENT = FALSE` (valeur par défaut), car un client distant non fiable pourrait se connecter sans authentification complémentaire

```
REVOKE EXECUTE ON utl_file, utl_http, utl_smtp FROM PUBLIC;
SELECT grantee FROM dba_role_privs WHERE granted_role = 'DBA'; -- utilisateurs ayant le role DBA
SELECT * FROM V$PWFILE_USERS; -- utilisateurs SYSDBA / SYSOPER
```

4.2 Les trois types d'audit

Type d'audit	Événements audités	Contenu de la trace
Standard (de la base)	Utilisation de privilèges, accès aux objets	Ensemble fixe de données
Basé sur les données (triggers)	Données modifiées par les instructions LMD	Défini par l'administrateur
Détaillé (FGA)	Instructions SQL (INSERT, UPDATE, DELETE, SELECT) selon le contenu	Ensemble fixe, incluant l'instruction SQL

4.3 Audit standard

Activé par le paramètre `AUDIT_TRAIL` :

Valeur	Effet
`NONE`	Aucune collecte
`OS`	Enregistrements dans la trace d'audit de l'OS (emplacement : `AUDIT_FILE_DEST`)
`DB`	Enregistrements dans la table `SYS.AUD\$`
`DB, EXTENDED`	Comme `DB`, avec en plus les colonnes `SQLBIND` et `SQLTEXT`
`XML`	Fichiers XML dans la trace d'audit de l'OS
`XML, EXTENDED`	Comme `XML`, avec `SQLBIND` et `SQLTEXT`

```
SHOW PARAMETER audit
ALTER SYSTEM SET audit_trail = DB SCOPE = SPFILE; -- avec un spfile (avec un pfile : editer le
    fichier)
```

Audit OS ou audit BD ? L'audit OS convient pour auditer plusieurs bases dans une même destination (un seul rapport), mais n'est pas offert par tous les OS (Unix). L'audit BD permet de consulter les résultats par de simples requêtes sur les vues du dictionnaire, mais `SYS.AUD\$` (tablespace SYSTEM) doit être archivée et purgée périodiquement, et protégée : un utilisateur malveillant pourrait y effacer ses actions. Solution : `AUDIT ALL ON SYS.AUD\$ BY ACCESS;`.

Quatre niveaux d'audit : commandes (LDD), privilèges système, objets de schéma (privilèges objet), connexions à la base.

```
-- Audit de commandes ou de privileges systeme
AUDIT {commande | privilege_systeme} [, ...] [BY user [, user] ...]
      [BY {SESSION | ACCESS}] [WHENEVER [NOT] SUCCESSFUL];
-- Audit d'objets
AUDIT commande [, commande] ... ON {[schema.]objet | DEFAULT}
      [BY {SESSION | ACCESS}] [WHENEVER [NOT] SUCCESSFUL];
```

`BY SESSION` : un seul enregistrement par session et par objet, quel que soit le nombre de commandes de même type. `BY ACCESS` : un enregistrement à chaque action (toujours le cas pour les commandes DDL). `WHENEVER [NOT] SUCCESSFUL` : audit seulement si la commande réussit (ou échoue).

```
AUDIT table; -- commandes, non cible
AUDIT TABLE BY hr WHENEVER NOT SUCCESSFUL; -- commandes, cible
AUDIT select any table, create any trigger; -- privileges systeme
AUDIT select any table BY hr BY SESSION;
AUDIT ALL ON hr.employees; -- privileges objet
AUDIT UPDATE, DELETE ON hr.employees BY ACCESS;
AUDIT session WHENEVER NOT SUCCESSFUL; -- connexions
```

Arrêt de l'audit (nécessite le privilège `AUDIT SYSTEM`) :

```
NOAUDIT session;
NOAUDIT session BY scott, lori;
NOAUDIT DELETE ANY TABLE;
NOAUDIT ALL; NOAUDIT ALL PRIVILEGES;
NOAUDIT DELETE ON emp;
NOAUDIT ALL ON emp;
```

Vues d'audit : `DBA_AUDIT_TRAIL` (toutes les entrées), `DBA_AUDIT_OBJECT` (objets de schéma), `DBA_AUDIT_SESSION` (connexions et déconnexions), `DBA_AUDIT_STATEMENT` (instructions).

4.4 Audit basé sur les données (triggers)

L'audit standard identifie les commandes exécutées sur une table mais pas *les modifications effectuées*. Pour garder trace des valeurs modifiées, on utilise des triggers.

- **Trigger DML** : enregistre les valeurs de toutes les modifications de la base (impossible avec l'audit intégré).
- **Trigger système** : audite créations, suppressions, connexions (inutile : l'audit intégré le fait).

```
CREATE [OR REPLACE] TRIGGER nom_trigger
  {BEFORE | AFTER | INSTEAD OF}
  {INSERT | UPDATE [OF col [, colN]] | DELETE} [OR ...]
  REFERENCING {[OLD [AS] ancien] | [NEW [AS] nouveau]}
  ON nom_table [FOR EACH ROW] [WHEN (condition)]
DECLARE /* declarations */
BEGIN /* traitement */
[EXCEPTION ...]
END;
```

4.5 Audit détaillé (FGA)

Le *Fine Grained Auditing* définit des conditions fines : il surveille l'accès aux données **selon leur contenu**, audite `SELECT` ou `INSERT / UPDATE / DELETE`, peut être lié à une table ou une vue, peut exécuter une procédure PL/SQL, et s'administre via le package `DBMS_FGA` (privilège `EXECUTE` requis).

Une **stratégie d'audit détaillé** se compose de deux parties : les *critères d'audit* (condition à vérifier) et l'*action d'audit* (procédure exécutée quand la condition est vérifiée). Elle est créée par

`ADD_POLICY`.

Sous-programme	Rôle
`ADD_POLICY`	Crée une stratégie avec le prédicat fourni comme condition
`DROP_POLICY`	Supprime une stratégie
`ENABLE_POLICY`	Active une stratégie
`DISABLE_POLICY`	Désactive une stratégie

```
CREATE OR REPLACE PROCEDURE ins_audit_message(p_schema VARCHAR2, p_table VARCHAR2, p_policy
    VARCHAR2) IS
BEGIN
  INSERT INTO audit_messages VALUES (SYSDATE, p_schema||'.'||p_table||'.'||p_policy);
END;
/
BEGIN
  DBMS_FGA.ADD_POLICY(
    object_schema => 'SCOTT', object_name => 'EMP',
    policy_name => 'DEPTNO20_SAL',
    audit_condition => 'DEPTNO = 20', audit_column => 'SAL',
    handler_schema => 'SYSTEM', handler_module => 'INS_AUDIT_MESSAGE',
    enable => TRUE);
END;
/
DBMS_FGA.ENABLE_POLICY (object_schema=>'hr', object_name=>'employees', policy_name=>'
    audit_emps_salary');
DBMS_FGA.DISABLE_POLICY(object_schema=>'hr', object_name=>'employees', policy_name=>'
    audit_emps_salary');
DBMS_FGA.DROP_POLICY (object_schema=>'hr', object_name=>'employees', policy_name=>'
    audit_emps_salary');
```

Vues :

- `DBA_FGA_AUDIT_TRAIL` : tous les événements d'audit détaillé ;
- `DBA_AUDIT_POLICIES` : toutes les stratégies de la base ; `ALL_AUDIT_POLICIES` : celles des objets accessibles ; `USER_AUDIT_POLICIES` : celles du schéma de l'utilisateur.

À retenir : règles de l'audit détaillé

Pour auditer toutes les instructions, utiliser une condition `NULL`. Ajouter une stratégie déjà existante provoque une erreur. La table ou la vue auditée doit déjà exister. Une condition invalide provoque une erreur à l'accès à l'objet audité. Si la colonne d'audit n'existe pas, aucune ligne n'est auditée.

5 Déplacement des données

Deux méthodes principales pour charger ou déplacer des données : **Oracle Data Pump** (export/import) et **SQL*Loader** (chargement de fichiers externes).

5.1 Oracle Data Pump

Introduit en **10g**, Data Pump remplace l'export/import classique, très lent (33 Go : 3h30 d'export, 7 à 8 h d'import). C'est un utilitaire *côté serveur* de déplacement rapide de données et de métadonnées entre bases Oracle, avec **traitement parallèle**, via les outils `expdp` et `impdp`.

Data Pump peut être lancé par Enterprise Manager, par les binaires `EXPDP` / `IMPDP` (dossier `bin` de `ORACLE_HOME`) ou par le package `SYS.DBMS_DATAPUMP`. Un **objet Directory** Oracle doit exister au préalable pour stocker les exports.

Mode	Paramètre	Contenu
Complet	`FULL`	Toute la base
Schéma	(mode par défaut) `SCHEMAS`	Un ou plusieurs schémas
Table	`TABLES`	Tables choisies dans un schéma
Tablespace	`TABLESPACES`	Tables d'un ou plusieurs tablespaces (les tablespaces eux-mêmes ne sont pas exportés)

Fichiers générés : fichiers **SQL** (instructions LDD de création des objets), fichiers **DUMP** (données exportées), fichiers **LOG** (journal du job).

Privilèges : `EXP_FULL_DATABASE` et `IMP_FULL_DATABASE` permettent d'exporter ou importer toute la base, un tablespace, un schéma ou une table d'un autre schéma. Les utilisateurs ayant le rôle DBA le peuvent.

Paramètre	Description
`CONTENT`	Données à décharger : `ALL`, `DATA_ONLY` ou `METADATA_ONLY`
`DIRECTORY`	Objet directory pour les fichiers dump et log
`DUMPFILE`	Liste des fichiers dump de destination
`EXCLUDE` / `INCLUDE`	Exclut / inclut des types d'objets (ex. `EXCLUDE=TABLE:EMP`)
`FILESIZE`	Taille maximale de chaque fichier dump (octets)
`FULL`, `SCHEMAS`, `TABLES`, `TABLESPACES`	Mode d'export / import
`TRANSPORT_TABLESPACES`	Tablespaces transportables (métadonnées)
`COMPRESSION`	Réduit la taille du dump (`METADATA_ONLY` ou `NONE`)
`ENCRYPTION_PASSWORD`	Mot de passe pour les colonnes chiffrées
`REMAP_TABLESPACE` (impdp)	Associe l'objet à un autre tablespace
`PARALLEL` (impdp)	Nombre de processus de travail actifs

L'aide s'obtient avec `expdp help=y` et `impdp help=y`. Le suivi d'un job se fait via la vue `DBA_DATAPUMP_JOBS`.

```
-- 1. Utilisateur et privileges
create user exp_imp identified by expimp;
grant connect, resource to exp_imp;
grant exp_full_database, imp_full_database to exp_imp;
-- 2. Repertoire Oracle (en tant que sysdba)
create or replace directory DATAPUMP as 'C:\oraclexe';
grant read, write on directory DATAPUMP to exp_imp;
-- 3. Export (en ligne de commande OS)
expdp exp_imp/expimp tables=exp_imp.test DIRECTORY=DATAPUMP DUMPFILE=test.dump LOGFILE=test.log
-- 4. Import des donnees seulement (ajoute les lignes une nouvelle fois)
impdp exp_imp/expimp content=DATA_ONLY tables=exp_imp.test DIRECTORY=DATAPUMP DUMPFILE=test.dump
    LOGFILE=test.log
```

Package `DBMS_DATAPUMP` (procédures principales) :

Procédure	Rôle
`OPEN`	Crée un nouveau job (opération `EXPORT` / `IMPORT`, mode, nom)
`ADD_FILE`	Spécifie un fichier dump, le log ou la sortie SQL (`filetype` : 1 = dump, 3 = log)
`METADATA_FILTER` / `DATA_FILTER`	Restreint les métadonnées / les données traitées
`METADATA_REMAP` / `METADATA_TRANSFORM`	Redirige / modifie les métadonnées
`SET_PARAMETER` / `SET_PARALLEL`	Modifie le traitement par défaut / le parallélisme
`START_JOB`	Démarre ou reprend un job
`STOP_JOB` / `WAIT_FOR_JOB`	Arrête un job / attend sa fin
`ATTACH` / `DETACH`	Se rattache à un job existant / s'en détache
`GET_STATUS`, `LOG_ENTRY`, `GET_DUMPFILE_INFO`	Statut du job, ajout au log, en-tête d'un fichier dump

```
DECLARE p_dph NUMBER;
BEGIN
  p_dph := DBMS_DATAPUMP.open(operation=>'EXPORT', job_mode=>'TABLE', job_name=>'DVP_EXPORT');
  DBMS_DATAPUMP.add_file(handle=>p_dph, filename=>'dvp.dmp', directory=>'DATAPUMP', filetype=>1);
  DBMS_DATAPUMP.add_file(handle=>p_dph, filename=>'tab_dvp.log', directory=>'DATAPUMP', filetype
      =>3);
  DBMS_DATAPUMP.metadata_filter(handle=>p_dph, name=>'SCHEMA_EXPR', value=>'IN (''DVP'')');
  DBMS_DATAPUMP.start_job(p_dph);
  DBMS_DATAPUMP.detach(p_dph);
END;
/
-- Pour un import : operation=>'IMPORT' et DBMS_DATAPUMP.set_parameter(handle=>p_dph,
-- name=>'TABLE_EXISTS_ACTION', value=>'REPLACE');
```

5.2 SQL*Loader

SQL*Loader charge des données de fichiers externes dans des tables Oracle ; son moteur d'analyse impose très peu de contraintes sur le format du fichier de données. Il s'appuie sur quatre fichiers :

Fichier	Contenu
Log file	Journal du chargement : noms des fichiers (control, bad, discard, données), champs et types chargés, messages d'erreur, nombre d'enregistrements lus, rejetés pour erreur ou pour critère de sélection, temps de chargement
Bad file	Enregistrements en erreur (non conformes au format du control file, violation de contrainte d'intégrité, tablespace plein…)
Discard file	Enregistrements écartés car ne vérifiant pas les conditions de sélection (`WHEN`) du control file
Control file	Décrit le chargement : emplacements des fichiers, mémoire, rejets, fichiers de données d'entrée, structure/types/longueurs, tables cibles, correspondance champs–colonnes, critères de sélection, format (délimité, taille fixe)

```
-- Exemple 1 : fichier delimite emp.dat (10001,"Scott Tiger",1000,40)
load data
infile 'c:\Emp.dat'
into table emp -- INSERT | REPLACE | TRUNCATE | APPEND
```

```
fields terminated by "," optionally enclosed by '"'
( empno, empname, sal, deptno )

-- Lancement
sqlldr user/password control=fichier.ctl log=fichier.log bad=fichier.bad discard=fichier.dsc
```

```
-- Exemple 2 : donnees a taille fixe, incluses dans le control file (infile *)
load data
infile *
append into table departments
( dept position (01:04) char(4),
  deptname position (05:25) char(20) )
begindata
COSC COMPUTER SCIENCE
ENGL ENGLISH LITERATURE

-- Exemple 3 : conversion au chargement (sequence, constante, expressions SQL)
load data infile 'Emp.dat' badfile 'bad.bad'
INTO TABLE modified_data
( rec_no "db_seq.nextval",
  region CONSTANT '31',
  time_loaded "to_char(SYSDATE, 'HH24:MI')",
  data1 position(1:5) ":data1/100",
  data2 position(6:15) "upper(:data2)",
  data3 position(16:22) "to_date(:data3, 'YYMMDD')" )

-- Exemple 4 : chargement selectif
load data infile 'emp.dat'
append into table emp
when (05) = 'B' and (15:20) = '990112'
trailing nullcols -- colonnes absentes considerees comme NULL
( empID POSITION(01:04), nom POSITION(05:14), date_emb POSITION(15:20) )

-- Exemple 5 : chargement dans plusieurs tables
LOAD DATA INFILE 'C:\emp.dat'
REPLACE INTO TABLE emp WHEN empno != ' '
( empno POSITION(1:4) INTEGER, ename POSITION(6:15) CHAR,
  deptno POSITION(17:18) CHAR, mgr POSITION(20:23) INTEGER )
INTO TABLE proj WHEN projno != ' '
( projno POSITION(25:27) INTEGER, empno POSITION(1:4) INTEGER )
```

6 Aide-mémoire

Thème	**À retenir**
Instance	SGA (shared pool, buffer cache, redo log buffer) + processus DBWn, LGWR, SMON, PMON, CKPT ; PGA privée par processus
Dictionnaire	Propriété de SYS ; vues `USER_`, `ALL_`, `DBA_`, `V\$`
Démarrage / arrêt	NOMOUNT → MOUNT → OPEN ; seul `SHUTDOWN ABORT` exige une récupération
Stockage	Tablespace ⊃ segment ⊃ extent ⊃ bloc ; fichier de données ↔ tablespace ; SYSTEM jamais offline
Sécurité utilisateur	Privilèges système/objet, rôles, profils ; `ADMIN OPTION` non cascadé, `GRANT OPTION` cascadé
Moindre privilège	`O7_DICTIONARY_ACCESSIBILITY`, `REMOTE_OS_AUTHENT` à FALSE ; révoquer `UTL_*` de PUBLIC
Audit	Standard (`AUDIT_TRAIL`, `AUDIT`/`NOAUDIT`), triggers (valeurs modifiées), FGA (`DBMS_FGA`)
Déplacement de données	Data Pump (`expdp`/`impdp`, 4 modes, directory) ; SQL*Loader (control, log, bad, discard)